CURSO DE FORMACIÓN · 20 HORAS

Big Data e IA

Bloque 6: Machine Learning aplicado



3 horas · scikit-learn · Google Colab

PLAN DE LA SESIÓN

Tres horas, tres partes

PARTE 1 · ORIENTATIVO 1 H

Preparar los datos

- El problema y la variable objetivo
- Distancia y ángulo
- Variables categóricas

PARTE 2 · ORIENTATIVO 1 H 15 MIN

Entrenar y evaluar

- Regresión logística
- Random Forest
- Matriz de confusión y comparación con StatsBomb

PARTE 3 · ORIENTATIVO 45 MIN

Ampliar y practicar

- Validación cruzada e importancia
- KMeans: zonas de remate
- Ejercicio

06

Machine Learning aplicado

Bloque 6 · 3 horas

6.1 · EL PROBLEMA

¿Cuál es la probabilidad de gol de un tiro?

Entrada

Dónde se tira, con qué parte del cuerpo, si había presión, la jugada.

Salida

Una probabilidad entre 0 y 1: el xG.

Para qué sirve

Medir la calidad de las ocasiones, más allá de si acabaron en gol.

La variable objetivo

```python
tiros["gol"] = (tiros["shot_outcome"] == "Goal").astype(int)

print(tiros["gol"].value_counts())
print(f"Porcentaje de goles: {tiros['gol'].mean() * 100:.1f}%")
```

Etiqueta

gol vale 1 si el tiro acabó en gol y 0 si no.

Desbalanceo

Alrededor de uno de cada diez tiros es gol: el accuracy engaña.

De coordenadas a geometría

python

```
dx = 120 - tiros["x"]
dy = tiros["y"] - 40

tiros["distancia"] = np.sqrt(dx**2 + dy**2)
# Ángulo bajo el que se ve la portería (8 m de ancho)
tiros["angulo"] = np.arctan2(8 * dx, dx**2 + dy**2 - 16)
```

Por qué

El modelo no entiende x = 108, y = 35, pero sí distancia y ángulo.

Portería

Está en x = 120, entre y = 36 e y = 44.

One-hot encoding

python

```python
base = tiros[["distancia", "angulo", "shot_body_part", "shot_type", "play_pattern"]].copy()
base["bajo_presion"] = tiros["under_pressure"].astype(int)
base["de_primeras"] = tiros["shot_first_time"].fillna(False).astype(int)

X = pd.get_dummies(base, columns=["shot_body_part", "shot_type", "play_pattern"],
                   drop_first=True)
y = tiros["gol"]
```

get_dummies

Convierte cada categoría en una columna de 0 y 1.

Entrenamiento y test

python

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

print(f"Entrenamiento: {len(X_train)} tiros ({y_train.sum()} goles)")
print(f"Test: {len(X_test)} tiros ({y_test.sum()} goles)")
```

stratify

Mantiene la proporción de goles en entrenamiento y en test.

random_state

Fija el azar para que todos obtengan el mismo reparto.

6.2 · MODELO 1

Regresión logística

```python
lr = make_pipeline(StandardScaler(),
                   LogisticRegression(max_iter=1000, class_weight="balanced"))
lr.fit(X_train, y_train)

prob_lr = lr.predict_proba(X_test)[:, 1]
print(f"AUC (regresión logística): {roc_auc_score(y_test, prob_lr):.3f}")
```

balanced

Da más peso a los goles, que son pocos.

AUC

0,5 es azar y 1 es perfecto.

6.2 · MODELO 2

Random Forest

python

```
rf = RandomForestClassifier(n_estimators=300, min_samples_leaf=5,
                            class_weight="balanced", random_state=42)
rf.fit(X_train, y_train)

prob_rf = rf.predict_proba(X_test)[:, 1]
print(f"AUC (Random Forest): {roc_auc_score(y_test, prob_rf):.3f}")
```

n_estimators

Número de árboles que votan.

min_samples_leaf

Evita árboles demasiado ajustados a pocos casos.

6.2 · EVALUACIÓN

Matriz de confusión y umbral

Falsos positivos

Dijo «gol» y no lo fue. Con umbral 0,5 son muchos.

Falsos negativos

Dijo «no gol» y lo fue: una ocasión que se escapó.

El umbral decide

¿Preferimos no dejar escapar ninguna buena ocasión o evitar falsas alarmas?

Con el umbral 0,5 el modelo da **muchos falsos positivos**: lo cambiamos según el coste de cada error.

Comparación con el xG de StatsBomb

Nuestro modelo

Con muy pocas variables y menos de mil tiros se acerca al profesional.

StatsBomb

Usa información que no tenemos, como la posición de los defensores.

Conclusión

La diferencia se explica sobre todo por la información disponible.

6.3 · FIABILIDAD

Validación cruzada

Idea

Repetir entrenamiento y test 5 veces con repartos distintos.

Qué vemos

El AUC cambia bastante de una vuelta a otra.

Conclusión

Con unos cien goles, desconfía de un único número.

Mira la **desviación**, no solo la media.

6.3 · INTERPRETAR

Qué variables pesan más

Cómo

rf.feature_importances_ ordena las variables según su influencia en el Random Forest.

Qué esperamos

Distancia y ángulo arriba: coincide con el sentido futbolístico.

Cuidado

Es importancia dentro del modelo, no causalidad.

6.3 · SIN ETIQUETA

Zonas de remate con KMeans

```python
km = KMeans(n_clusters=4, n_init=10, random_state=42)
tiros["zona"] = km.fit_predict(tiros[["x", "y"]])

resumen = tiros.groupby("zona").agg(tiros=("gol", "size"),
                                    goles=("gol", "sum"),
                                    xg_medio=("shot_xg", "mean")).round(3)
```

Sin etiqueta

El modelo agrupa solo con la posición; después miramos cuántos goles hay en cada zona.

¿Merece la pena una red?

En tabla y con pocos datos

Un Random Forest suele rendir igual o mejor.

Explicabilidad

Es más fácil explicar un árbol que una red.

Cuándo sí

Imágenes, texto y audio, con muchos datos.

EJERCICIO · 30 MINUTOS

Ahora te toca a ti

1. Cambia random_state en train_test_split y observa cuánto varía el AUC. ¿Qué te dice sobre la fiabilidad?
2. Entrena el modelo sin distancia y angulo. ¿Cuánto empeora?
3. Prueba otro umbral (0,3 y 0,7). ¿Cómo cambian recall y precisión de «Gol»?
4. ¿Qué otras variables añadirías si las tuvieras? (Pista: ¿dónde estaban los defensores?)

CIERRE

Lo que nos llevamos

- Un proyecto de ML empieza por la pregunta y los datos, no por el algoritmo.
- Las variables bien construidas (distancia y ángulo) valen más que un modelo complicado.
- Con pocos datos, un único número no es fiable.
- Comparar con una referencia profesional enseña cuánto nos falta.

SIGUIENTE BLOQUE

Proyecto integrador

Cada alumno une todo lo aprendido en un mini proyecto y lo presenta.